

Noisy Soft Syndrome Decoding of Quantum LDPC Codes for the Quantum Erasure Channel

Gayathri R.^{†*}, Shobhit Bhatnagar^{^*} and P Vijay Kumar[†]

[†]Department of Electrical Communication Engineering, IISc Bangalore

[^]Department of Electrical and Computer Engineering, The University of Arizona
{gayatr,shobhitb97,pvk1729}@gmail.com

Abstract—Erasure decoding of quantum low-density parity-check (QLDPC) codes has recently received a lot of interest as qubit erasures occur in many physical systems such as neutral-atom systems, trapped-ion systems etc. However, decoding of QLDPC codes heavily relies on syndrome information, which may be unreliable as syndrome extraction circuits are prone to errors in practice. While syndrome errors are usually modeled as flipping of a binary syndrome bit, typically, measurement outcomes yield analog voltage/current values, which are then discretized to obtain the binary syndrome. Prior work on correcting qubit errors in the presence of syndrome noise shows that this soft (i.e., analog) information can be exploited to achieve performance improvement. In this paper, we consider the case of qubit erasures and present an iterative decoding algorithm, based on the hardware-friendly min-sum algorithm, that incorporates this soft information to perform check node updates. We show via simulations on hypergraph product codes and lifted product codes that our decoder achieves performance very close to that in the case of noiseless syndrome for a certain noise regime.

I. INTRODUCTION

Quantum computation promises to provide an exponential advantage over classical computation, but quantum information is much more fragile than its classical counterpart. Quantum error correction (QEC) is therefore a necessary step in building a practical quantum computer. A promising class of quantum error-correcting codes to achieve practical fault tolerance is the family of quantum low density parity-check (QLDPC) codes, which are analogous to classical LDPC codes [1]. A significant amount of effort has been made over the last decade to construct QLDPC codes that have both high rate and large minimum distance. This effort has led to many non-trivial code constructions [2]–[9]. One such construction is the hypergraph product (HGP) construction by Tillich and Zémor [10], and its subsequent generalization, the lifted product (LP) construction by Pantaleev and Kalachev [11].

Analogous to classical LDPC codes, QLDPC codes are equipped with efficient message-passing decoding algorithms, such as the belief propagation algorithm and the hardware-friendly min-sum algorithm (MSA) [12]. However, contrary to the classical case, the input to the decoder of a QLDPC code is a syndrome vector instead of a received vector. The decoder is tasked with finding an error estimate that corresponds to this syndrome. An interesting and significant difference from the classical case here is the phenomenon of degeneracy, where the decoder may output any one among a set of error estimates rather than a single, unique error vector.

* indicates equal contribution.

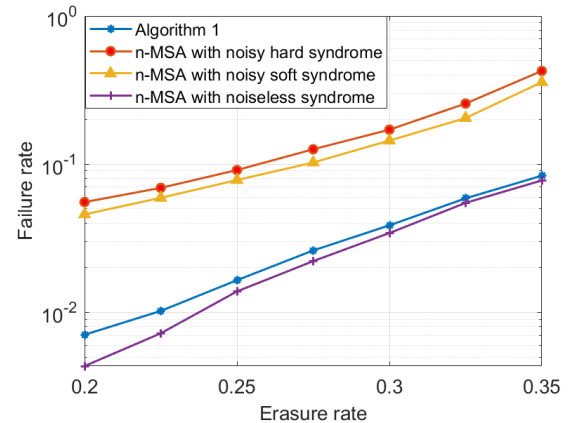


Fig. 1: Performance comparison of Algorithm 1 in this paper, the n-MSA algorithm with noisy hard syndrome, the n-MSA algorithm with noisy soft syndrome and the n-MSA algorithm with noiseless syndrome, for the $[[882, 24, 18 \leq d \leq 24]]$ B1 Lifted Product code. Here, noise standard deviation $\sigma = 0.4$.

The syndrome extraction process, however, is itself prone to errors. Thus, the QLDPC decoder may receive as input an erroneous syndrome, in which case it will try to find an error estimate that corresponds to this erroneous syndrome. While syndrome measurements can be repeated, this process is inefficient and emphasis in the literature has been on single-shot error correction [13], where decoding is performed using only a single round of (noisy) syndrome measurements. Usually, syndrome errors are modeled as a flipping of the binary syndrome value with some probability [14]. For such a setting, one can employ quantum data-syndrome (QDS) codes [15], which recover from syndrome errors by coding across the syndrome bits. However, Pattison et al. in [14] point out that in practice, syndrome measurements yield analog voltage/current values which are then discretized to obtain the binary syndrome. Then in the case of surface codes they modify the Minimum Weight Perfect Matching and Union Find decoders to incorporate this analog soft information and achieve improved performance. In a follow-up work, Raveendran et al. [16] borrow the Gaussian noise model for syndrome errors from [14] and modify the MSA algorithm to incorporate the noisy soft syndrome. For a certain noise regime, this algorithm achieves performance almost identical to that in the case of noiseless syndrome, for various families of QLDPC codes.

The focus in both [14] and [16] is on recovering from qubit errors in the presence of a noisy soft syndrome. On the

other hand, qubit erasures occur in many physical systems such as neutral-atom systems [17], photonic systems [18], [19], trapped-ion systems [20], superconducting qubits [21] etc. Due to this, erasure decoding of QLDPC codes has recently received a lot of attention in the literature [22]–[24]. However, in all these works, the idealized setting of a noiseless syndrome is assumed.

Our Contributions: In this paper, we focus on erasure decoding of QLDPC codes in the presence of a noisy soft syndrome. We assume the Gaussian noise model for syndrome noise assumed in [14], [16] and present an iterative decoder that iterates between performing the MSA algorithm on a sub-Tanner graph consisting of check nodes whose associated reliability is larger than a suitable threshold and are therefore reliable, and updating the reliabilities of less reliable check nodes using final messages accumulated at variable nodes. As the algorithm proceeds, more check nodes become reliable and participate in the message-passing. We demonstrate the efficacy of our decoder by showing through simulations on HGP and LP codes that, for a certain noise regime, it achieves performance very close to that of the noiseless syndrome setting. Fig. 1 shows the performance of our decoder for the $[[882, 24, 18 \leq d \leq 24]]$ LP code [11].

Organization of the paper: Section II reviews the stabilizer formalism used in QEC and outlines the syndrome-based MSA decoding approach for QLDPC codes over the quantum erasure channel. It also briefly introduces the classes of HGP and LP codes, the syndrome noise model and the quantum erasure channel. In Section III, we detail our proposed decoding algorithm and Section IV presents the corresponding simulation results. Finally, Section V offers concluding remarks.

II. PRELIMINARIES

Notation: We use I_n to denote the $(n \times n)$ identity matrix. We use $\mathbb{F}_2$ to denote the finite field of two elements. For a vector $E \in \mathbb{F}_2^n$, we use E_i to denote the i -th coordinate of E , and $\text{supp}(E)$ to denote the support of E , i.e., $\text{supp}(E) = \{i \mid E_i \neq 0, i \in \{1, 2, \dots, n\}\}$. We use $A \otimes B$ to denote the tensor product of two matrices A and B .

A. Stabilizer Formalism of QEC

A qubit, the fundamental unit of quantum information, is associated with a 2-dimensional vector in the complex Hilbert space $\mathbb{C}^2$. The Pauli group on n qubits is defined as $\mathcal{P}_n = \{\omega P_1 \otimes P_2 \otimes \dots \otimes P_n\}$, where $\omega \in \{\pm 1, \pm i\}$, where $i^2 = -1$, and $P_i \in \{I_2, X, Y, Z\}$ for each $i \in \{1, 2, \dots, n\}$. Here,

$$X = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}, Y = \begin{bmatrix} 0 & -i \\ i & 0 \end{bmatrix}, Z = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$$

are the single-qubit Pauli matrices. An $[[n, k]]$ stabilizer code [25] is a 2^k -dimensional subspace of the Hilbert space $\mathbb{C}^{2^n}$ that encodes k logical qubits into n physical qubits. It is defined as the simultaneous $+1$ -eigenspace of an abelian subgroup $\mathcal{S}$ of $\mathcal{P}_n$ that does not contain $-I_{2^n}$.

Calderbank-Shor-Steane (CSS) codes [26], [27] are a popular class of stabilizer codes where each stabilizer is a tensor product of either only I_2 -type and X -type Pauli matrices, or only I_2 -type and Z -type Pauli matrices. Consider

two classical binary codes $\mathcal{C}_X$ and $\mathcal{C}_Z$ with corresponding parity-check matrices H_X and H_Z , respectively, such that $\mathcal{C}_Z^\perp \subseteq \mathcal{C}_X$. It follows that $H_X H_Z^T = 0$. The X -type stabilizers for the corresponding CSS code are given by

$$\mathcal{S}_X = \{X^{a_1} \otimes X^{a_2} \otimes \dots \otimes X^{a_n} \mid [a_1, a_2, \dots, a_n] \in \mathcal{C}_X^\perp\},$$

and the Z -type stabilizers are given by

$$\mathcal{S}_Z = \{Z^{b_1} \otimes Z^{b_2} \otimes \dots \otimes Z^{b_n} \mid [b_1, b_2, \dots, b_n] \in \mathcal{C}_Z^\perp\}.$$

An X -type (Z -type) error is a Pauli error that consists of only I_2 -type and X -type (Z -type) Pauli matrices. Independent X -type and Z -type errors can be handled separately (and analogously) in the CSS framework. H_X is used to correct Z -type errors while H_Z is used to correct X -type errors. The syndrome for X -type errors is given by $s_X^T = H_Z E_X^T$, where E_X is the corresponding error vector for the X -type Pauli error. E_X contains I_2 -type and X -type Pauli matrices in its vector representation. In the remainder of this paper, we will deal with only X -type errors, and therefore use the less cumbersome notation s for s_X , H for H_Z , and E for E_X , so that $s^T = H E^T$. The i -th bit s_i of the syndrome s is 0 if E commutes with the i -th Z -type stabilizer generator, and 1 if E anti-commutes with it.

B. Quantum LDPC Codes

QLDPC codes are quantum analogues of classical LDPC codes and correspond to the case when the H_X and H_Z matrices mentioned before are sparse in nature. We briefly describe below two popular classes of QLDPC codes known as HGP codes [10] and LP codes [11].

For HGP codes, the matrices H_X and H_Z are given by

$$H_X = (H_1 \otimes I_{n_2} \quad I_{m_1} \otimes H_2^T), \\ H_Z = (I_{n_1} \otimes H_2 \quad H_1^T \otimes I_{m_2}),$$

where $H_1 \in \mathbb{F}_2^{m_1 \times n_1}$ and $H_2 \in \mathbb{F}_2^{m_2 \times n_2}$ are parity check matrices of two classical binary LDPC codes.

LP codes are generalizations of HGP codes, where each scalar entry in the matrices H_X and H_Z above is replaced by a circulant matrix. This is called *lifting*. Thus, the matrices H_X and H_Z for an LP code are given by

$$H_X = (\tilde{H}_1 \otimes I_{n_2} \quad I_{m_1} \otimes \tilde{H}_2^T), \\ H_Z = (I_{n_1} \otimes \tilde{H}_2 \quad \tilde{H}_1^T \otimes I_{m_2}),$$

where $\tilde{H}_1$ and $\tilde{H}_2$ are $(m_1 \times n_1)$ and $(m_2 \times n_2)$ matrices, respectively, whose entries are $(L \times L)$ binary circulant matrices.

C. The Quantum Erasure Channel

In the quantum erasure channel model, each qubit is lost or erased independently with a certain probability. The loss of a qubit can be detected, and an erased qubit is replaced with a maximally mixed state

$$\frac{I_2}{2} = \begin{bmatrix} \frac{1}{2} & 0 \\ 0 & \frac{1}{2} \end{bmatrix},$$

implying that each erased qubit can be interpreted as being acted upon by the Pauli matrices I_2 , X , Y , and Z with equal probability. Thus, the problem of erasure correction is converted into that of correcting errors with known locations.

D. Noiseless Syndrome Decoding of QLDPC Codes

The MSA algorithm is a low-complexity message-passing algorithm that passes messages along the edges of the Tanner graph associated with a code. Let $H \in \mathbb{F}_2^{m \times n}$ be a parity-check matrix of a binary $[n, k \geq (n - m)]$ code $\mathcal{C}$. The corresponding Tanner graph $\mathcal{T}(H) = (V, C, \mathcal{E})$ is a bipartite graph, where $V = \{1, 2, \dots, n\}$ is the set of variable nodes corresponding to the n code symbols, $C = \{1, 2, \dots, m\}$ is the set of check nodes corresponding to the rows of H , and $\mathcal{E}$ is the set of edges of $\mathcal{T}(H)$. Check node i is connected to variable node j iff $H_{i,j} = 1$, where $H_{i,j}$ denotes the (i, j) -th entry of H . The neighborhood $\mathcal{N}(v)$ of a variable node $v \in V$ is the set of all the check nodes that are connected to it and similarly the neighborhood $\mathcal{N}(c)$ of a check node $c \in C$ is the set of all the variable nodes connected to it.

The messages exchanged between check and variable nodes in the MSA algorithm are in the form of log-likelihood ratios (LLRs). Consider a channel where X -type errors occur with probability p . The channel LLR for a variable node $v \in V$ is given by

$$\lambda_v = \ln\left(\frac{1-p}{p}\right).$$

As mentioned above, an erased qubit is replaced with a maximally mixed state $\frac{I_2}{2}$, which corresponds to the value $p = \frac{1}{2}$ [22]. Thus, the channel LLR associated with erased variable nodes is 0. The channel LLR associated with unerased variable nodes is $+\infty$ as the corresponding value of p is 0. As stated before, by replacing each erased qubit with a maximally mixed state, the problem of erasure recovery is converted to that of correcting errors with known locations. Let E denote this error vector. The task of the MSA algorithm is to find an error estimate $\hat{E}$ such that the corresponding estimated syndrome $\hat{s}^T = H\hat{E}^T$ is equal to the received (noiseless) syndrome $s^T = HE^T$. However, due to degeneracy [28], decoding is successful only if the Pauli operator corresponding to $(E + \hat{E})$ belongs to the stabilizer group $\mathcal{S}$, otherwise, we encounter a logical error. If the decoder is unable to find an error-estimate $\hat{E}$ such that $s^T = H\hat{E}^T$, we say that the decoder has failed due to non-convergence.

At the 0^{th} iteration, $t = 0$, of the MSA algorithm, every variable node $v \in V$ passes the message $\nu_{v \rightarrow c}^{(0)}$ to each check node $c \in \mathcal{N}(v)$, where

$$\nu_{v \rightarrow c}^{(0)} = \lambda_v.$$

At the t^{th} iteration, $t \geq 1$, every check node c passes the message $\mu_{c \rightarrow v}^{(t)}$ to each variable node $v \in \mathcal{N}(c)$, where

$$\mu_{c \rightarrow v}^{(t)} = (-1)^{s_c} \left(\prod_{v' \in \mathcal{N}(c) \setminus \{v\}} \text{sgn}(\nu_{v' \rightarrow c}^{(t-1)}) \right) \left(\min_{v' \in \mathcal{N}(c) \setminus \{v\}} |\nu_{v' \rightarrow c}^{(t-1)}| \right), \quad (1)$$

where $\text{sgn}(\cdot)$ denotes the signum function which maps a real number x to $+1$ if $x > 0$, and to -1 if $x \leq 0$. Recall that s_c denotes the c^{th} bit of the syndrome vector s . Subsequently,

each variable node v passes the message $\nu_{v \rightarrow c}^{(t)}$ to each check node $c \in \mathcal{N}(v)$, where

$$\nu_{v \rightarrow c}^{(t)} = \lambda_v + \sum_{c' \in \mathcal{N}(v) \setminus \{c\}} \mu_{c' \rightarrow v}^{(t)}. \quad (2)$$

Thus, apart from the 0^{th} iteration, each iteration of the MSA algorithm consists of two rounds of message-passing: first from check nodes to variable nodes, and then from variable nodes to check nodes. It has been observed in the literature that normalizing the check to variable node messages in (1) by a suitable positive constant $\alpha < 1$ improves performance [12]. In this paper, we employ this variant of the MSA algorithm, which is called the normalized MSA (n-MSA) algorithm.

After T iterations of message-passing, the final message at variable node v is computed as

$$\nu_v = \lambda_v + \sum_{c' \in \mathcal{N}(v)} \mu_{c' \rightarrow v}^{(T)}.$$

The absolute value of ν_v is referred to as the reliability of the variable node v . To get the error estimate $\hat{E}$, hard decisions are taken on the variable nodes as

$$\hat{E}_v = \begin{cases} 0, & \nu_v > 0, \\ 1, & \nu_v \leq 0. \end{cases}$$

We wish to emphasize here a major difference between the cases of noisy and noiseless syndromes, which is the halting condition for the decoder. When syndromes are noiseless, the decoder halts if it finds an error estimate that matches the received syndrome. However, when the received syndrome can be erroneous, this is clearly not a good halting criterion. Thus, as also observed in [16], coming up with a good halting condition is a key consideration when designing a decoder for the setting of noisy syndromes. We will address this issue when we describe our decoding algorithm later.

E. Gaussian Syndrome Noise Model

We adopt the Gaussian syndrome noise model in [14], [16]. In case of perfect, noiseless syndrome measurements, the measurement outcome yields the real value $\bar{s}_i = +1$ if the Pauli error commutes with the i -th stabilizer, and $\bar{s}_i = -1$ if the Pauli error anti-commutes with it. The binary syndrome bit s_i is obtained as

$$s_i = \begin{cases} 0, & \text{if } \bar{s}_i = +1, \\ 1, & \text{if } \bar{s}_i = -1. \end{cases}$$

In the presence of Gaussian noise, the measurement outcome yields a noisy value $r_i = \bar{s}_i + n_i$, where n_i follows the Gaussian distribution $n_i \sim \mathcal{N}(0, \sigma^2)$ for each i , and all the n_i 's are independent. It follows that the distribution of r_i conditioned on $\bar{s}_i$ is given by $(r_i | \bar{s}_i = +1) \sim \mathcal{N}(+1, \sigma^2)$, and $(r_i | \bar{s}_i = -1) \sim \mathcal{N}(-1, \sigma^2)$. The LLR for the i -th component of the noisy soft syndrome is then given by

$$\gamma_i = \log \frac{\Pr(r_i | \bar{s}_i = 1)}{\Pr(r_i | \bar{s}_i = -1)} = \frac{2r_i}{\sigma^2}.$$

The noisy, hard (binary) syndrome bit s_i is then obtained as

$$s_i = \begin{cases} 0, & \text{sgn}(\gamma_i) = 1, \\ 1, & \text{sgn}(\gamma_i) = -1. \end{cases}$$

The magnitude of γ_i is referred to as the reliability of s_i (or alternatively sometimes as the reliability of check node i).

III. SOFT SYNDROME ERASURE DECODING

We will now discuss our soft syndrome decoding algorithm, the pseudocode for which is presented in Algorithm 1. Consider the input Tanner graph $\mathcal{T}_1(H) = (V_1, C, \mathcal{E}_1)$. As in [28], we initialize the channel LLR of unerased variable nodes to $\text{llr}_{\max} = 25$ (instead of $+\infty$) and that of erased variable nodes to $\text{llr}_{\min} = 10^{-5}$ (instead of 0) for numerical stability. Let $C_E \subseteq C$ denote the set of check nodes that have at least one erased variable node in their neighborhood. It follows that the true (i.e., noiseless) syndrome bit corresponding to each of the remaining check nodes in $C \setminus C_E$ must be 0. Thus, we update γ_c to $+\infty$ for each check node $c \in C \setminus C_E$.

To perform message-passing, we use a modified Tanner graph (see Fig. 2) as in [16], where, to each check node $c \in C$, an additional type of variable node v_c , which we will refer to as a virtual variable node, is attached, while retaining the existing variable nodes, check nodes, and edges of $\mathcal{T}_1(H)$. We will denote the modified Tanner graph by $\mathcal{T}_2(H) = (V_1 \cup V_2, C, \mathcal{E}_2)$, where V_2 denotes the set of virtual variable nodes. The channel LLR associated with virtual variable node v_c is set to γ_c , and is denoted by γ_{v_c} in lieu of λ_{v_c} as a reminder that this variable node is a virtual variable node. Thus, there is a direct correspondence between reliabilities of check nodes of the original Tanner graph $\mathcal{T}_1(H)$ and reliabilities of the corresponding virtual variable nodes of the modified Tanner graph $\mathcal{T}_2(H)$. Informally, the syndrome values on check nodes of the original Tanner graph have been “pushed onto” the virtual variable nodes in the modified Tanner graph. Thus, in the modified Tanner graph, the sum of the variable node bit values in the neighborhood of a check node should match the bit value on the corresponding virtual variable node, which implies that the syndrome bit associated to each check node in the modified Tanner graph should be 0.

We now describe the message-passing schedule for $\mathcal{T}_2(H)$. First, we choose a threshold Γ and denote by $C^{(a)}$ the set of all check nodes c for which $|\gamma_{v_c}| > \Gamma$. We denote by $\mathcal{T}_2(H)|_{C^{(a)}}$ the subgraph of $\mathcal{T}_2(H)$ obtained by removing all the check nodes in $C \setminus C^{(a)}$ and all the edges incident to any such check node. *We remark here that the main idea behind Algorithm 1 is the empirical observation that message-passing decoding is more robust to disregarding a few check nodes than to the presence of even a small number of erroneous syndromes.* For example, for the $[[882, 24, 18 \leq d \leq 24]]$ LP code in Fig. 1, at erasure rate 0.25, the n-MSA algorithm with noiseless syndrome has failure rate 1.38×10^{-2} . If 15 check nodes are removed randomly then the failure rate increases to around 1.76×10^{-2} , whereas in the presence of 5 erroneous syndrome bits the failure rate is around 2.2×10^{-1} . So our algorithm excludes a few low-reliability check nodes at each iteration and updates them eventually as per a schedule. In a nutshell, we will perform message passing only on $\mathcal{T}_2(H)|_{C^{(a)}}$ for T iterations and use the messages accumulated at variable nodes to update the virtual variable nodes corresponding to check nodes in $C^{(b)} = C \setminus C^{(a)}$ at the $(T+1)^{\text{th}}$ iteration. We thus get

an estimate $\hat{E}$ of the error on the variable nodes and an estimate $\hat{s}$ of the syndrome on the virtual variable nodes and check if $H\hat{E}^T = \hat{s}^T$. If not, we will update $C^{(a)}$ by thresholding the final messages at the virtual variable nodes and again perform the n-MSA algorithm on $\mathcal{T}_2(H)|_{C^{(a)}}$. As the algorithm proceeds, the size of $C^{(b)}$ reduces which helps the n-MSA algorithm to converge. Note that if Γ is too small, then many low-reliability virtual variable nodes, which could potentially be erroneous, will participate in message-passing. On the other hand, if Γ is too large, very few check nodes from $\mathcal{T}_2(H)$ will remain in $\mathcal{T}_2(H)|_{C^{(a)}}$. Thus, the value of Γ must be carefully chosen.

We now elaborate the steps of Algorithm 1. Upon determining $C^{(a)}$, we perform the n-MSA algorithm on $\mathcal{T}_2(H)|_{C^{(a)}}$ for T iterations. At the 0th iteration, $t = 0$, every variable node $v \in V_1$ passes the message $\nu_{v \rightarrow c}^{(0)}$ to each check node $c \in \mathcal{N}(v)$, where

$$\nu_{v \rightarrow c}^{(0)} = \lambda_v,$$

and every virtual variable node $v_c \in V_2$ passes the message $\nu_{v_c \rightarrow c}^{(0)}$ to its corresponding check node c , where

$$\nu_{v_c \rightarrow c}^{(0)} = \gamma_{v_c}.$$

For $t \geq 1$, according to the message-passing rules of the n-MSA algorithm described in Section II, check node c passes the message $\mu_{c \rightarrow v}^{(t)}$ to each $v \in \mathcal{N}(c)$ (including virtual variable node v_c), where

$$\mu_{c \rightarrow v}^{(t)} = (+1) \left(\prod_{v' \in \mathcal{N}(c) \setminus \{v\}} \text{sgn}(\nu_{v' \rightarrow c}^{(t-1)}) \right) \left(\min_{v' \in \mathcal{N}(c) \setminus \{v\}} |\nu_{v' \rightarrow c}^{(t-1)}| \right).$$

Observe that the term $(-1)^{s_c}$ indicating the sign of the syndrome bit in (1) is absorbed in the product term above, since the $\text{sgn}(\cdot)$ term corresponding to the virtual variable node v_c carries this sign. Furthermore, the message $\mu_{c \rightarrow v}^{(t)}$ also incorporates the reliability of the noisy syndrome bit s_c . The update rules from variable to check nodes remain the same as in (2) explained in the previous section.

After completing T iterations of the n-MSA algorithm on $\mathcal{T}_2(H)|_{C^{(a)}}$, we update virtual variable nodes in $\{v_c \mid c \in C^{(b)}\}$ with the following final message:

$$\nu_{v_c} = \gamma_{v_c} + \mu_{c \rightarrow v_c}^{(T)}, \quad (3)$$

where

$$\mu_{c \rightarrow v_c}^{(T)} = \left(\prod_{v' \in \mathcal{N}(c) \setminus \{v_c\}} \text{sgn}(\nu_{v' \rightarrow c}^{(T-1)}) \right) \left(\min_{v' \in \mathcal{N}(c) \setminus \{v_c\}} |\nu_{v' \rightarrow c}^{(T-1)}| \right).$$

The schedule showing the two steps of message-passing is illustrated in the Fig. 2. We then take hard decisions on all variable and virtual variable nodes and check if for each $c \in C$, the sum of the hard decisions on all its neighbors is 0. In other words, if $\hat{E}$ denotes the vector of hard decisions on variable nodes (i.e., $\hat{E}$ is the estimated error) and $\hat{s}$ denotes the vector of hard decisions on virtual variable nodes (i.e.,

$\hat{s}$ is the estimated syndrome), we check if $H\hat{E}^T = \hat{s}^T$. If this is true, the decoder halts and outputs $\hat{E}$ as the estimated error vector. If $H\hat{E}^T \neq \hat{s}^T$, we update the sets $C^{(a)}$ and $C^{(b)}$ by thresholding the absolute value of the final messages accumulated at virtual variable nodes corresponding to each check node, and restart the message-passing on the updated $\mathcal{T}_2(H)|_{C^{(a)}}$ with the schedule described above.

The decoder iterates between performing the n-MSA algorithm and updating $C^{(a)}$ until either we get $H\hat{E}^T = \hat{s}^T$, or a maximum number of predetermined iterations T' is reached. We wish to emphasize here that the message-passing schedule of our algorithm as well as the syndrome update rules are significantly different from that in [16], where syndrome updates as well as halting condition check is performed after each message-passing iteration. The algorithm in [16] was proposed for qubit errors and performs close to the noiseless syndrome case. However, for the case of qubit erasures this algorithm does not perform as well. For example, for the $[[882, 24, 18 \leq d \leq 24]]$ LP code in Fig. 1, at erasure rate 0.25, the algorithm in [16] has failure rate 1.7×10^{-1} for $\sigma = 0.3$, which is far from the noiseless syndrome case (1.38×10^{-2}). However, as Fig. 1 shows, Algorithm 1 has performance very close to the noiseless syndrome case even when $\sigma = 0.4$.

We remark here that when the noise standard deviation is large, it is possible that check nodes with high reliabilities are also erroneous. In this case, one can employ an additional check if the halting condition is not met and the maximum number of iterations T' has been reached. One can check whether $\hat{s}^T$ matches $H\hat{E}^T$ for all virtual variable nodes that have absolute value of the final message greater than a suitably chosen threshold value $\Gamma' > \Gamma$. In the simulation results presented in Fig. 1 and Fig. 3, we employ this additional check.

IV. SIMULATION RESULTS

We simulate our algorithm for the $[[2025, 81]]$ HGP code obtained from the GitHub repository in [22] and the $[[882, 24, 18 \leq d \leq 24]]$ LP code obtained from Appendix B in [11], which we will refer to as the B1 code. In all the simulations, the normalization constant $\alpha = 0.61$ is used to perform the n-MSA algorithm, and $T = 25$ iterations of message-passing are performed. Further, as in [28], we limit the values of the messages passed from variable nodes to check nodes to the range $[-25, 25]$.

Fig. 1 shows the performance comparison between Algorithm 1, the n-MSA algorithm with noisy hard syndrome, the n-MSA algorithm with noisy soft syndrome and the n-MSA algorithm with noiseless syndrome for the B1 code. The n-MSA algorithm with noisy soft syndrome is nothing but the regular message-passing on the Tanner graph $\mathcal{T}_2(H) = (V_1 \cup V_2, C, \mathcal{E}_2)$ where the set of check nodes $C^{(a)}$ is fixed to C . On the Y-axis we plot failure rate, which includes failures due to both logical errors and non-convergence. The value of σ is 0.4 and each curve is generated by performing 5×10^4 trials. The simulation parameters for Algorithm 1 are $T' = 10$, $\Gamma = 2.5$.

Fig. 3 shows performance plots similar to Fig. 1 for the $[[2025, 81]]$ HGP code. The value of σ is 0.35. The simulation parameters for Algorithm 1 are $T' = 10$, $\Gamma = 1$.

Algorithm 1 Noisy Soft Syndrome Decoding Algorithm

Input: Tanner graph $\mathcal{T}_1(H) = (V_1, C, \mathcal{E}_1)$, Set $V_E \subset V_1$ of erased variable nodes, Set C_E of check nodes connected to at least one erased variable node, Reliabilities $\{\gamma_c \mid c \in C\}$ of check nodes.

Output: Error estimate $\hat{E}$ or non-convergence.

```

1: for  $v \in V_1$  do
2:   if  $v \in V_E$  then
3:      $\lambda_v = \text{llr}_{\min}$ 
4:   else
5:      $\lambda_v = \text{llr}_{\max}$ 
6:   end if
7: end for
8:  $\gamma_c \leftarrow +\infty \forall c \in C \setminus C_E$ 
9: Get modified Tanner graph  $\mathcal{T}_2(H) = (V_1 \cup V_2, C, \mathcal{E}_2) \triangleright$ 
    $V_2 = \{v_c \mid c \in C\}$  is the set of virtual variable nodes
10:  $C^{(a)} \leftarrow \{c \in C \mid |\gamma_{v_c}| > \Gamma\}$ 
11:  $C^{(b)} \leftarrow C \setminus C^{(a)}$ 
12: for  $v \in V_1 \cup V_2$  do
13:    $\nu_{v \rightarrow c}^{(0)} \leftarrow \lambda_v$  for all  $c \in \mathcal{N}(v)$ 
14:    $\nu_{v_c \rightarrow c}^{(0)} \leftarrow \gamma_{v_c}$ 
15: end for
16: while iter  $\leq T'$  do
17:   perform n-MSA on  $\mathcal{T}_2(H)|_{C^{(a)}}$  for  $T$  iterations
18:   update virtual variable nodes  $\{v_c \mid c \in C^{(b)}\}$  as in
   (3)
19:    $\hat{E} \leftarrow$  hard values of variable nodes in  $V_1$ 
20:    $\hat{s} \leftarrow$  hard values of virtual variable nodes in  $V_2$ 
21:   if  $(\hat{s}^T = H\hat{E}^T)$  then
22:     return  $\hat{E}$ 
23:   else
24:      $C^{(a)} \leftarrow \{c \in C \mid |\nu_{v_c}| > \Gamma\}$ 
25:      $C^{(b)} \leftarrow C \setminus C^{(a)}$ 
26:   end if
27:   iter++
28: end while
29: return non-convergence

```

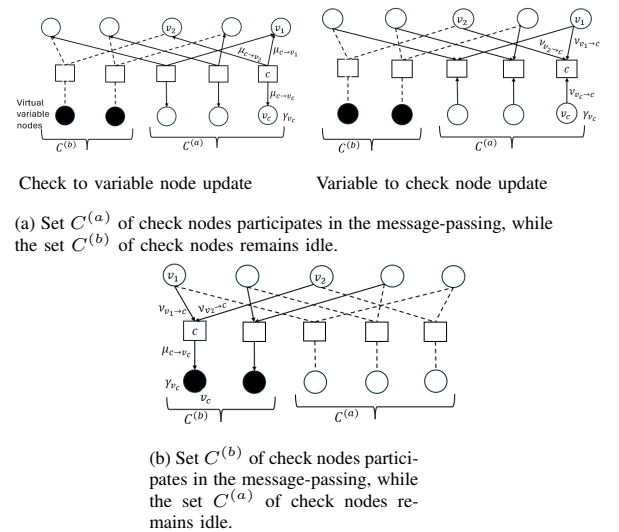


Fig. 2: Schedule of the message-passing for Algorithm 1.

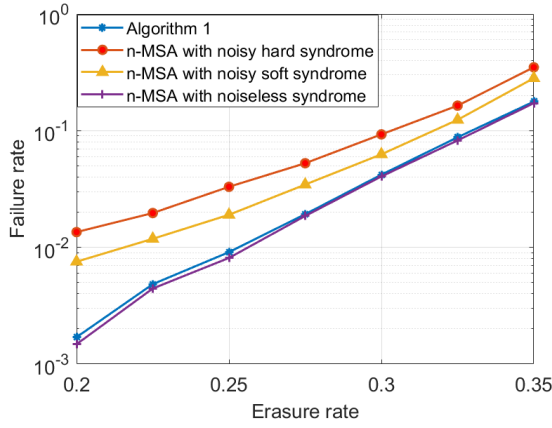


Fig. 3: Performance comparison of Algorithm 1, n-MSA algorithm with noisy hard syndrome, n-MSA algorithm with noisy soft syndrome and n-MSA algorithm with noiseless syndrome for the $[[2025, 81]]$ HGP code. Here, $\sigma = 0.35$.

As mentioned in Section III, for generating the curve corresponding to Algorithm 1 in both Fig. 1 and Fig. 3, we use an additional check: if $H\hat{E}^T \neq \hat{s}^T$ after T' iterations, we check whether the match happens for all virtual variable nodes that have an absolute value of the final message greater than $\Gamma' = 10$. If this is the case, the decoder outputs $\hat{E}$ as the error estimate, otherwise, it outputs non-convergence.

We now discuss the complexity of Algorithm 1. The complexity of performing the n-MSA algorithm for T iterations is $O(nT)$, where a popular choice of T is $\log n$ [29]. Thus, the overall complexity of Algorithm 1 is $O(nT' \log n)$. As our simulations suggest, T' can be chosen to be a small number, and thus, in the typical case, the complexity of Algorithm 1 is similar to that of the n-MSA algorithm.

V. CONCLUSION

In this paper, we present an erasure decoder for QLDPC codes for the setting of a noisy soft syndrome. The iterative decoder leverages the soft information from the noisy syndrome to classify the check nodes, enabling their selective participation in message-passing according to a defined schedule. The algorithm alternates between running the n-MSA algorithm on the sub-Tanner graph and reclassifying the check nodes until the halting condition is met. We showed through simulations on HGP and LP codes that our decoding algorithm achieves performance close to the noiseless syndrome case for a certain noise regime. Handling degeneracy to further improve decoding performance is an interesting direction for future work.

REFERENCES

- [1] R. Gallager, "Low-density parity-check codes," *IRE Transactions on Information Theory*, vol. 8, no. 1, pp. 21–28, 1962.
- [2] M. B. Hastings, J. Haah, and R. O'Donnell, "Fiber bundle codes: breaking the $n^{1/2} \text{polylog}(n)$ barrier for quantum LDPC codes," in *Proceedings of the 53rd Annual ACM SIGACT Symposium on Theory of Computing*, 2021, pp. 1276–1288.
- [3] N. P. Breuckmann and J. N. Eberhardt, "Balanced product quantum codes," *IEEE Transactions on Information Theory*, vol. 67, no. 10, pp. 6653–6674, 2021.
- [4] P. Panteleev and G. Kalachev, "Quantum LDPC codes with almost linear minimum distance," *IEEE Transactions on Information Theory*, vol. 68, no. 1, pp. 213–229, 2021.

- [5] —, "Asymptotically good quantum and locally testable classical LDPC codes," in *Proceedings of the 54th Annual ACM SIGACT Symposium on Theory of Computing*, 2022, pp. 375–388.
- [6] A. Leverrier and G. Zémor, "Quantum Tanner codes," in *2022 IEEE 63rd Annual Symposium on Foundations of Computer Science (FOCS)*. IEEE, 2022, pp. 872–883.
- [7] —, "Decoding quantum Tanner codes," *IEEE Transactions on Information Theory*, vol. 69, no. 8, pp. 5100–5115, 2023.
- [8] I. Dinur, M.-H. Hsieh, T.-C. Lin, and T. Vidick, "Good quantum LDPC codes with linear time decoders," in *Proceedings of the 55th annual ACM symposium on theory of computing*, 2023, pp. 905–918.
- [9] S. Gu, C. A. Pattison, and E. Tang, "An efficient decoder for a linear distance quantum LDPC code," in *Proceedings of the 55th Annual ACM Symposium on Theory of Computing*, 2023, pp. 919–932.
- [10] J.-P. Tillich and G. Zémor, "Quantum LDPC codes with positive rate and minimum distance proportional to the square root of the blocklength," *IEEE Transactions on Information Theory*, vol. 60, no. 2, pp. 1193–1202, 2013.
- [11] P. Panteleev and G. Kalachev, "Degenerate quantum LDPC codes with good finite length performance," *Quantum*, vol. 5, p. 585, 2021.
- [12] J. Chen, A. Dholakia, E. Eleftheriou, M. P. Fossorier, and X.-Y. Hu, "Reduced-complexity decoding of LDPC codes," *IEEE transactions on communications*, vol. 53, no. 8, pp. 1288–1299, 2005.
- [13] E. T. Campbell, "A theory of single-shot error correction for adversarial noise," *Quantum Science and Technology*, vol. 4, no. 2, p. 025006, 2019.
- [14] C. A. Pattison, M. E. Beverland, M. P. da Silva, and N. Delfosse, "Improved quantum error correction using soft information," *arXiv preprint arXiv:2107.13589*, 2021.
- [15] A. Ashikhmin, C.-Y. Lai, and T. A. Brun, "Quantum data-syndrome codes," *IEEE Journal on Selected Areas in Communications*, vol. 38, no. 3, pp. 449–462, 2020.
- [16] N. Raveendran, J. Valls, A. K. Pradhan, N. Rengaswamy, F. Garcia-Herrero, and B. Vasić, "Soft syndrome iterative decoding of quantum LDPC codes and hardware architectures," *EPJ Quantum Technology*, vol. 10, no. 1, p. 45, 2023.
- [17] Y. Wu, S. Kolkowitz, S. Puri, and J. D. Thompson, "Erasure conversion for fault-tolerant quantum computing in alkaline earth Rydberg atom arrays," *Nature communications*, vol. 13, no. 1, p. 4657, 2022.
- [18] E. Knill, R. Laflamme, and G. J. Milburn, "A scheme for efficient quantum computation with linear optics," *nature*, vol. 409, no. 6816, pp. 46–52, 2001.
- [19] S. Bartolucci, P. Birchall, H. Bombin, H. Cable, C. Dawson, M. Gimeno-Segovia, E. Johnston, K. Kielsing, N. Nickerson, M. Pant *et al.*, "Fusion-based quantum computation," *Nature Communications*, vol. 14, no. 1, p. 912, 2023.
- [20] M. Kang, W. C. Campbell, and K. R. Brown, "Quantum error correction with metastable states of trapped ions using erasure conversion," *PRX Quantum*, vol. 4, no. 2, p. 020358, 2023.
- [21] A. Kubica, A. Haim, Y. Vaknin, H. Levine, F. Brandão, and A. Retzker, "Erasure qubits: Overcoming the T_1 limit in superconducting circuits," *Physical Review X*, vol. 13, no. 4, p. 041022, 2023.
- [22] N. Connolly, V. Londe, A. Leverrier, and N. Delfosse, "Fast erasure decoder for hypergraph product codes," *Quantum*, vol. 8, p. 1450, 2024.
- [23] K.-Y. Kuo and Y. Ouyang, "Degenerate quantum erasure decoding," *arXiv preprint arXiv:2411.13509*, 2024.
- [24] B. Freire, N. Delfosse, and A. Leverrier, "Optimizing hypergraph product codes with random walks, simulated annealing and reinforcement learning (2025)," *arXiv preprint arXiv:2501.09622*.
- [25] D. Gottesman, *Stabilizer codes and quantum error correction*. California Institute of Technology, 1997.
- [26] A. R. Calderbank and P. W. Shor, "Good quantum error-correcting codes exist," *Physical Review A*, vol. 54, no. 2, p. 1098, 1996.
- [27] A. M. Steane, "Error correcting codes in quantum theory," *Physical Review Letters*, vol. 77, no. 5, p. 793, 1996.
- [28] H. Yao, W. A. Laban, C. Häger, A. G. i Amat, and H. D. Pfister, "Belief propagation decoding of quantum LDPC codes with guided decimation," in *2024 IEEE International Symposium on Information Theory (ISIT)*. IEEE, 2024, pp. 2478–2483.
- [29] J. Du Crest, M. Mhalla, and V. Savin, "Stabilizer inactivation for message-passing decoding of quantum LDPC codes," in *2022 IEEE Information Theory Workshop (ITW)*. IEEE, 2022, pp. 488–493.